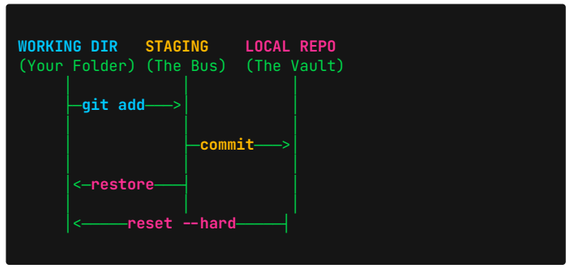
GIT & GITHUB SURVIVAL WALL

// ANDY'S EDITION V1.0

Version Control • Collaboration • Branching • Disaster Recovery

1. THE MENTAL MODEL

Git has 4 isolated zones. You must move files between them explicitly.



+ REMOTE REPO (GitHub) via push/pull

2. START & CLONE

GIT CLONE
`git clone git@github.com:user/repo.git`
Downloads an existing remote repo to your machine.
Use when: Joining a project or copying from GitHub.

GIT INIT
`git init`
Turns your current folder into a Git repository.
Use when: Starting a brand new local project.

3. THE DAILY CORE LOOP

GIT STATUS (THE RADAR)
`git status`
Shows what is modified, staged, or untracked.
RUN THIS BEFORE AND AFTER EVERY OTHER COMMAND.

GIT ADD (STAGE)
`git add analysis.py`
Adds a specific file to the staging area (The Bus).
`git add .`
Adds ALL changed files.

GIT COMMIT (SAVE)
`git commit -m "Fix data cleaning bug"`
Takes a permanent snapshot of the staging area.
Rule: Messages must be imperative ("Fix", not "Fixed").

GIT PUSH (UPLOAD)
`git push origin main`
Uploads your local commits to GitHub (Remote).

4. HISTORY & INSPECTION

GIT LOG
`git log --oneline --graph`
Shows a clean, visual tree of your commit history.
Press 'q' to exit the log view.

GIT DIFF
`git diff`
Shows exact line-by-line changes in unstaged files.
`git diff --staged`
Shows changes that are staged (ready to commit).

5. BRANCHING (THE SANDBOX)

Never code on main. Create branches for experiments. If the experiment fails, delete the branch.

GIT BRANCH
`git branch`
Lists all local branches. Active branch has a star (*).

GIT SWITCH / CHECKOUT
`git switch -c feature-model-v2`
Creates a NEW branch and switches to it instantly.
`git switch main`
Switches back to the main branch safely.
Note: `checkout -b` does the exact same thing as `switch -c`.

GIT BRANCH -D (DELETE)
`git branch -d feature-model-v2`
Deletes the branch locally (must be on another branch).

6. SYNCING & MERGING

GIT PULL (DOWNLOAD & MERGE)
`git pull origin main`
Fetches changes from GitHub and merges them into your current branch.
Use when: Teammates updated the repo while you slept.

GIT MERGE
`git merge feature-model-v2`
Takes the changes from the feature branch and fuses them into your CURRENT branch.

7. MERGE CONFLICTS

Don't panic. Git pauses the merge because you and a teammate edited the **exact same line**.

```
<<<<<<< HEAD (Current Branch)
learning_rate = 0.01
=======
learning_rate = 0.005
>>>>>>> feature-tune (Incoming)
```

How to fix it: 1. Open the file in VS Code.
2. Delete the markers (<<<<, ====, >>>>).
3. Keep the code you want (or combine both).
4. git add [file]
5. git commit -m "Resolve conflict"

ABORT MERGE (PANIC BUTTON)
`git merge --abort`
Cancels the merge and puts everything back to normal.

8. EMERGENCY RECOVERY ZONE

"I MESSED UP, HOW DO I UNDO?"

1. "I STAGED A FILE BY ACCIDENT!"
`git restore --staged file.py`
Removes it from Staging, but keeps your code safe.

2. "I RUINED MY FILE. RESET IT!"
`git restore file.py`
Deletes all uncommitted changes. Restores last save.
⚠️ DANGER: Your recent typing is lost forever.

3. "TYPO IN THE LAST COMMIT MESSAGE!"
`git commit --amend -m "New message"`
Replaces the last commit message. Do NOT do this if already pushed!

4. "UNDO MY LAST COMMIT, KEEP CODE"
`git reset --soft HEAD~1`
Deletes the commit, but puts all files back into Staging.

5. "NUKE IT. DELETE EVERYTHING."
`git reset --hard HEAD~1`
Deletes the commit AND the code changes forever.

9. GITHUB COLLABORATION

How Data Science teams work together without overwriting each other's code.

1. ENSURE YOU ARE UP TO DATE
`git checkout main` `git pull`

2. CREATE YOUR ISOLATED BRANCH
`git switch -c add-xgboost-model`

3. WORK, ADD, AND COMMIT
`git add model.py` `git commit -m "Add XGBoost"`

4. PUSH BRANCH TO GITHUB
`git push -u origin add-xgboost-model`
The `-u` links your local branch to the remote one.

5. PULL REQUEST (PR)
Go to GitHub.com, click "Compare & Pull Request".
Teammates review it, then it gets merged into main.

10. GIT MANTRAS

COMMIT EARLY.
COMMIT OFTEN.

NEVER PUSH DIRECTLY
TO MAIN.

GIT STATUS IS YOUR
BEST FRIEND.
RUN IT BEFORE YOU PANIC.

A COMMIT IS A SAVE POINT.
A BRANCH IS AN ALTERNATE UNIVERSE.